Data Structures and Algorithms I.



University of Szeged

Robert Nagy

2025/26. Spring Semester



Sorting Algorithms



Why is sorting important?



Database systems

Searching

Naive approach
$O(n)$

Binary search
$O(\log_2(n))$

Naive search (Brute force)

Task: Find ***3***

7, 4, 5, 9, 2, 3, 0, 1, 8

Naive search (Brute force)

Task: Find ***3***

7, 4, 5, 9, 2, 3, 0, 1, 8





Worst case scenario?

Binary search

Task: Find ***3***

7, 4, 5, 9, 2, 3, 0, 1, 8

Binary search

Step 1: Sort the input

0, 1, 2, 3, 4, 5, 7, 8, 9

Binary search

Step 2: Find the midpoint

0, 1, 2, 3, 4, 5, 7, 8, 9



Binary search

Step 3: Cut the search space in half

0, 1, 2, 3

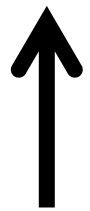
Binary search

Step 4: Repeat steps 1-3 until the key is found

0, 1, 2, 3

Binary search

0, 1, 2, 3



Binary search

3



F.Y.I - Watching security footage for stolen items

- Binary search!
  - Start at the middle of the footage
  - If the item is present, move to the ¾ marker
  - If not, move to the ¼ marker
  - You will find the exact time the item was stolen in $\log_2 n$ time



But how quickly can we sort?

Different approaches to sorting

- Comparison based
- Non comparison based
- Hybrid

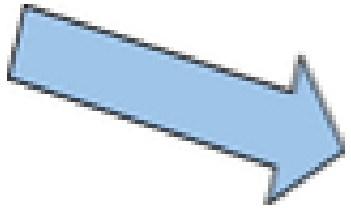
Read more: https://www.geeksforgeeks.org/dsa/sorting-algorithms/

Stability

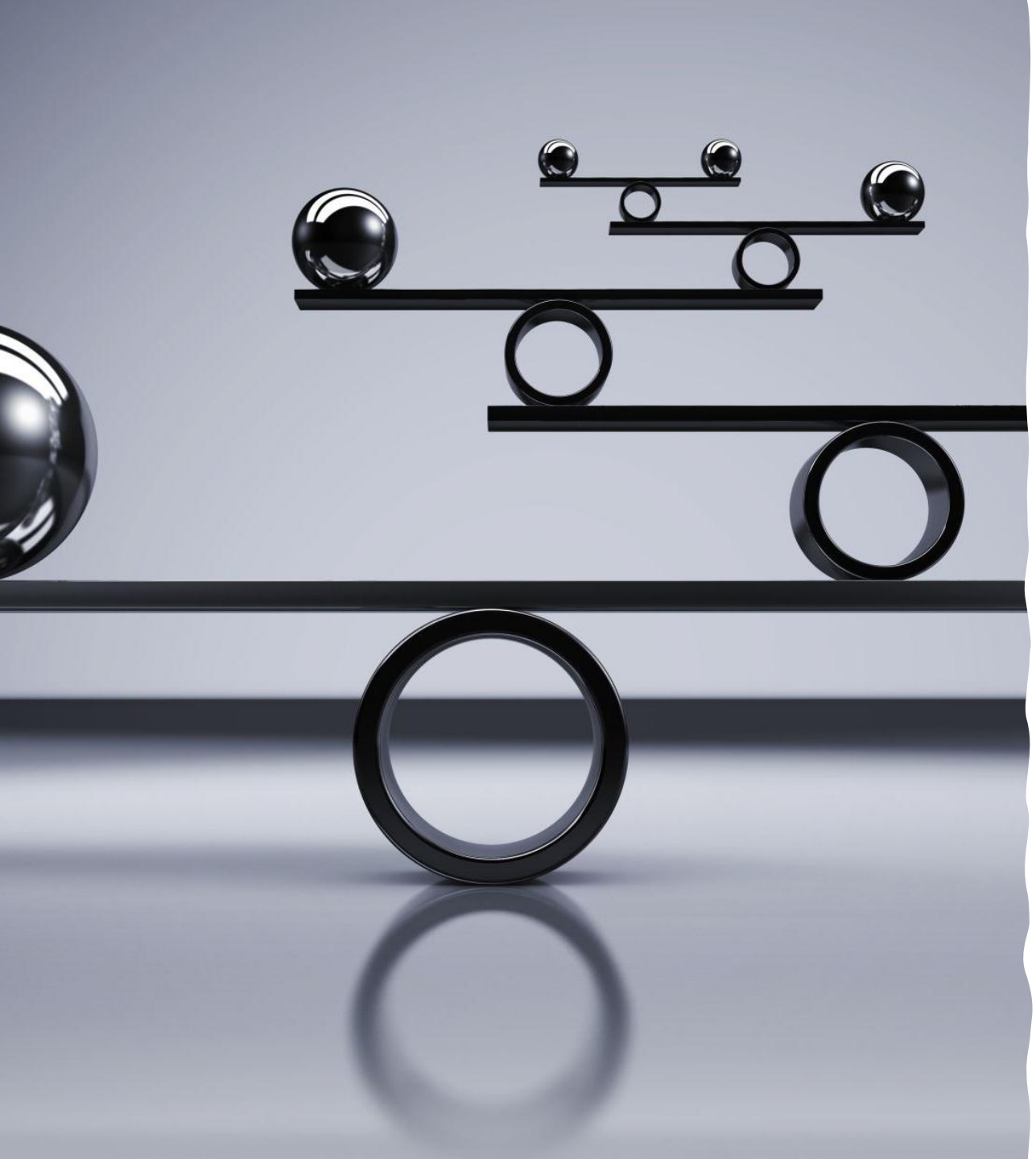
Stable sorting

Unstable sorting

BEFORE	
Name	**Grade**
Dave	C
Earl	B
Fabian	B
Gill	B
Greg	A
Harry	A



AFTER	
Name	**Grade**
Greg	A
Harry	A
Gill	B
Fabian	B
Earl	B
Dave	C



Comparison based sorting

We have already seen

Insertion sort - $O(n^2)$

Merge sort – $O(n * \log(n))$

Insertion sort – F.Y.I

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $key = A[i]$	c_2	$n-1$
3 // Insert $A[i]$ into the sorted subarray $A[1:i-1]$.	0	$n-1$
4 $j = i-1$	c_4	$n-1$
5 **while** $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6 $A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n}(t_i - 1)$
7 $j = j-1$	c_7	$\sum_{i=2}^{n}(t_i - 1)$
8 $A[j+1] = key$	c_8	$n-1$

Merge sort – F.Y.I

```
MERGE-SORT(A, p, r)
1  if p ≥ r                                  // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                           // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)                       // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)                   // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //      copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //      remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```

Characteristics

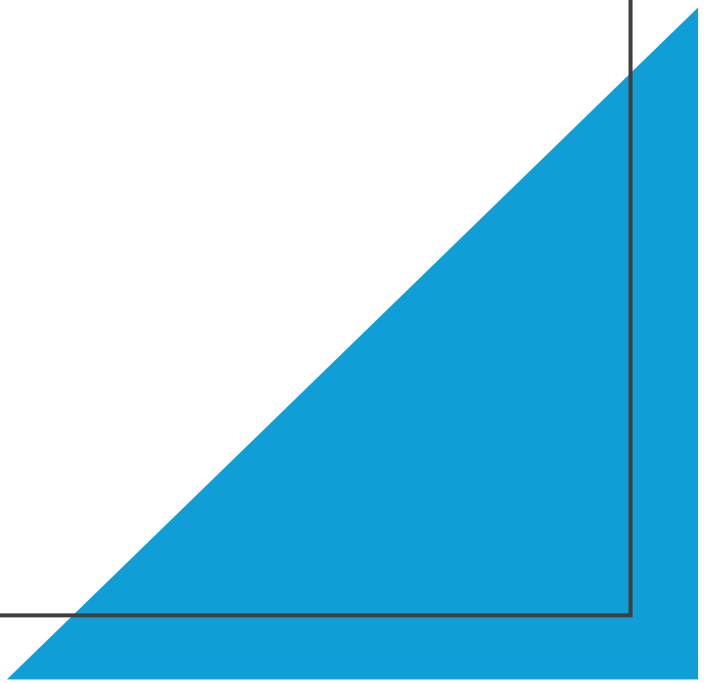
Insertion sort

- Efficient for small datasets
- Low memory usage
- Stable

Merge sort

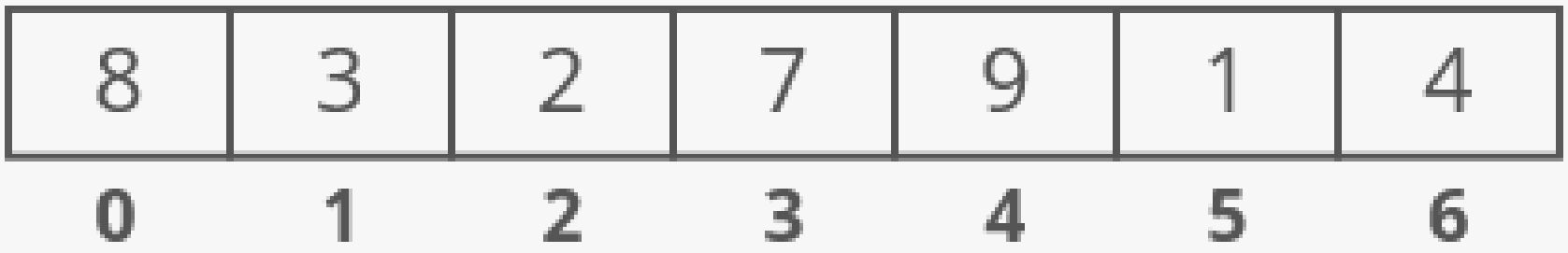
- Efficient for large datasets
- Uses extra memory
- Stable

Heapsort

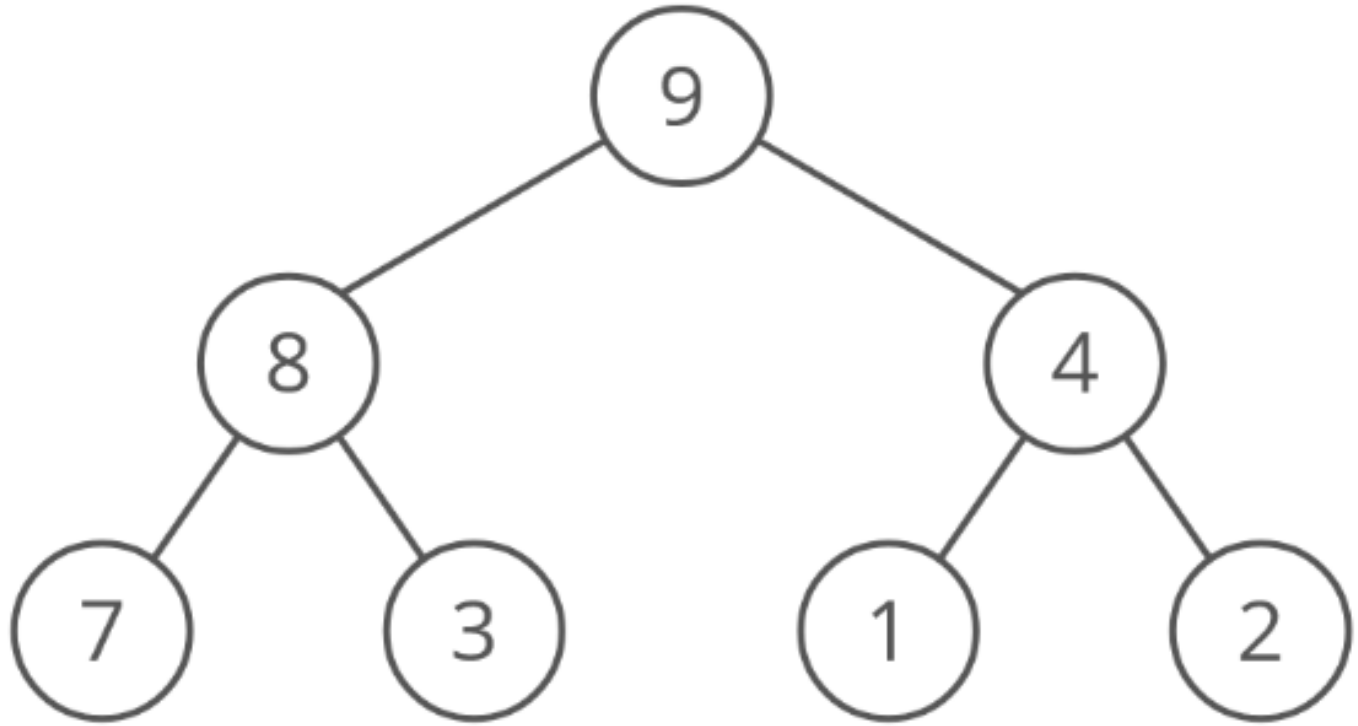


Heapsort

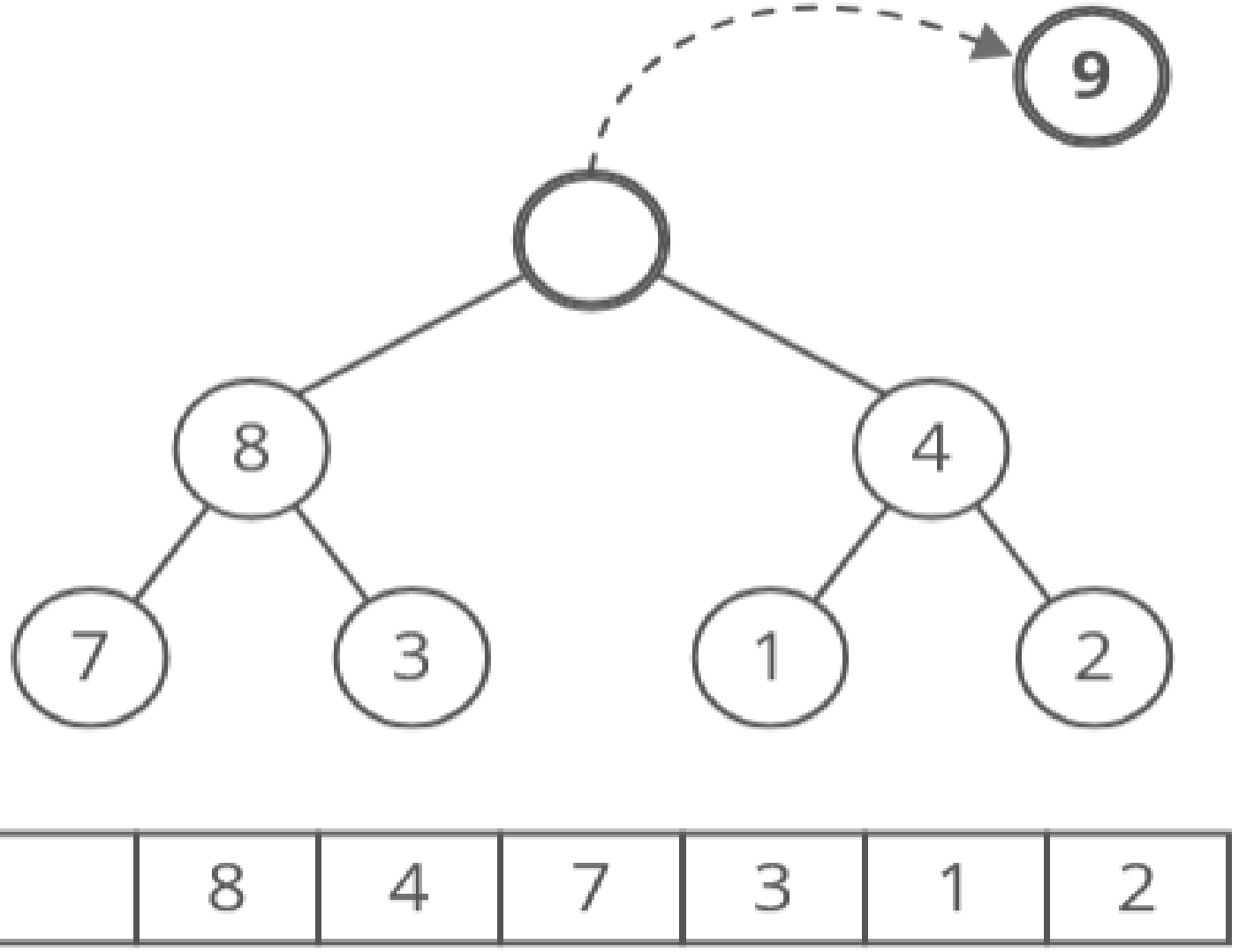
- Unstable
- Fast – O($n * log(n)$)
- Does not use extra memory



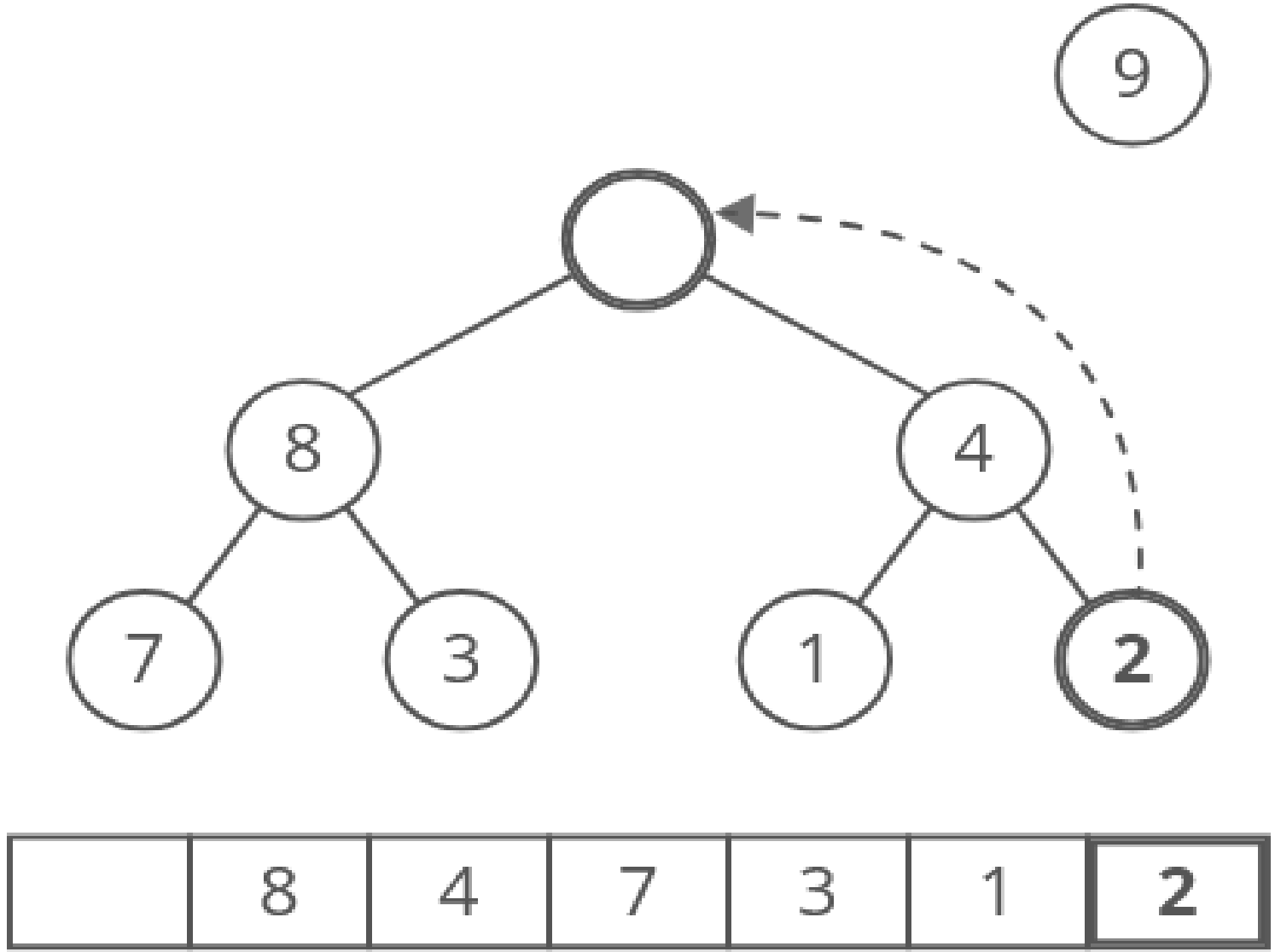
https://www.interviewcake.com/concept/java/heapsort

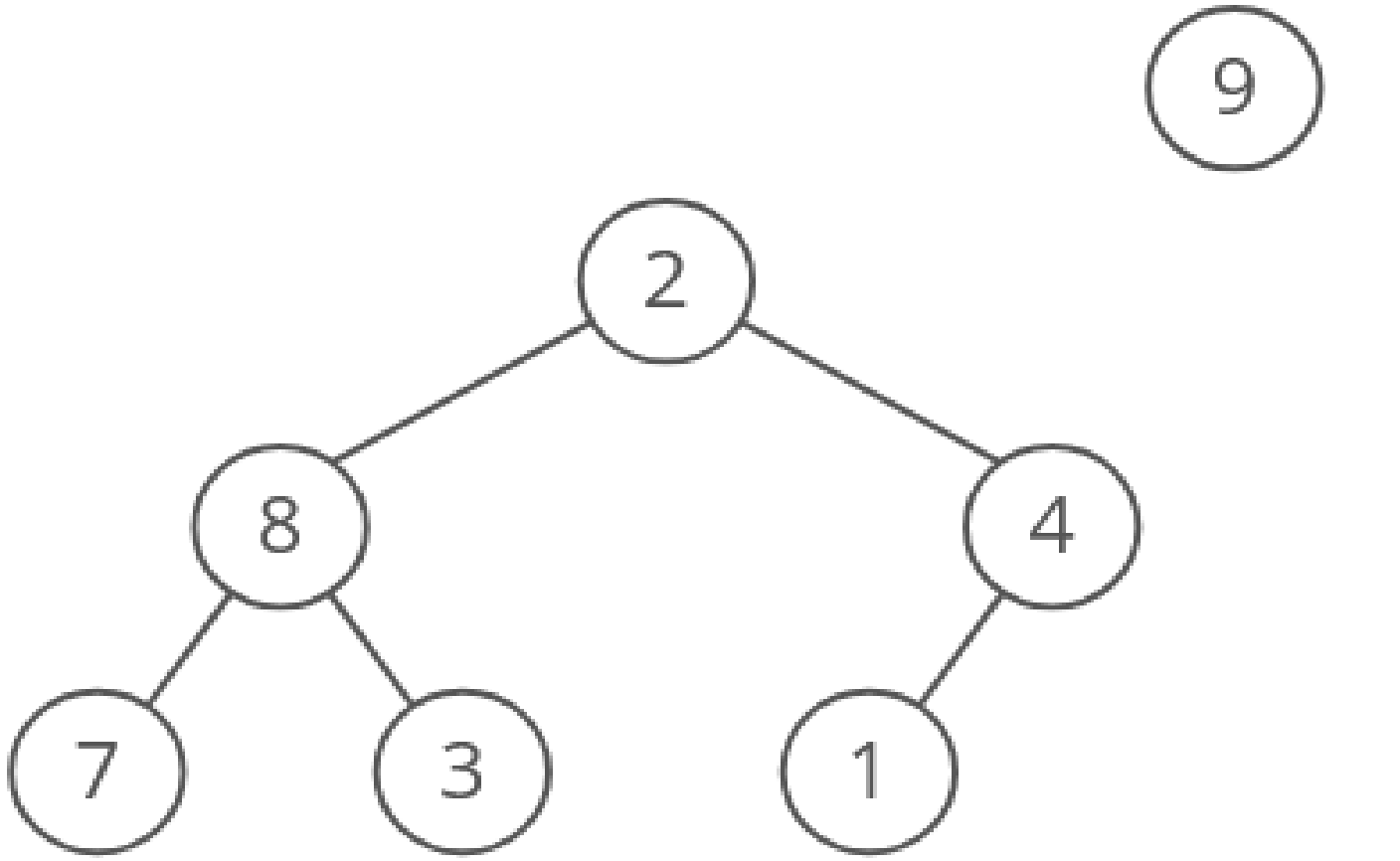

9
8
4
7
3
1
2


9
8
4
7
3
1
2

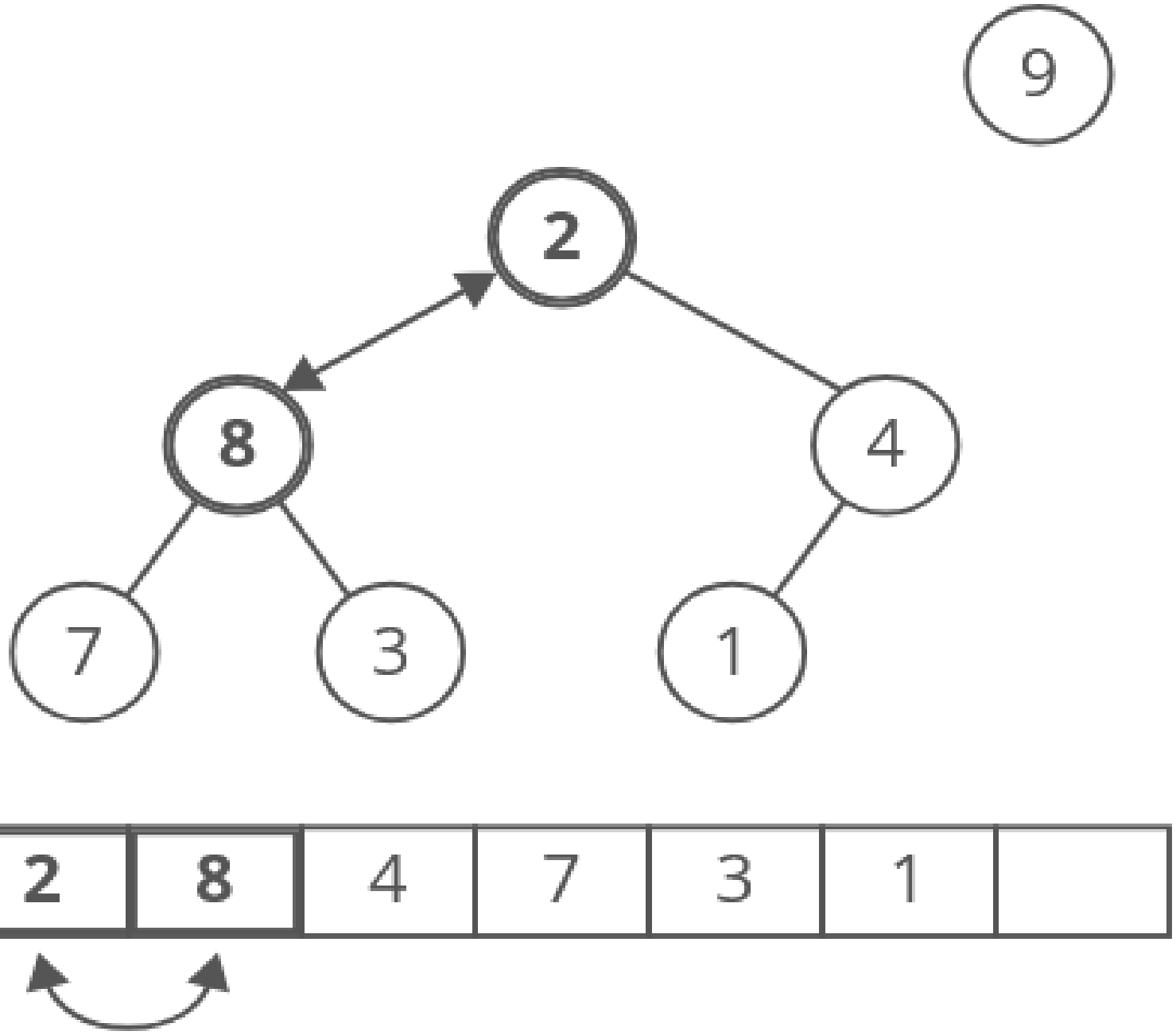


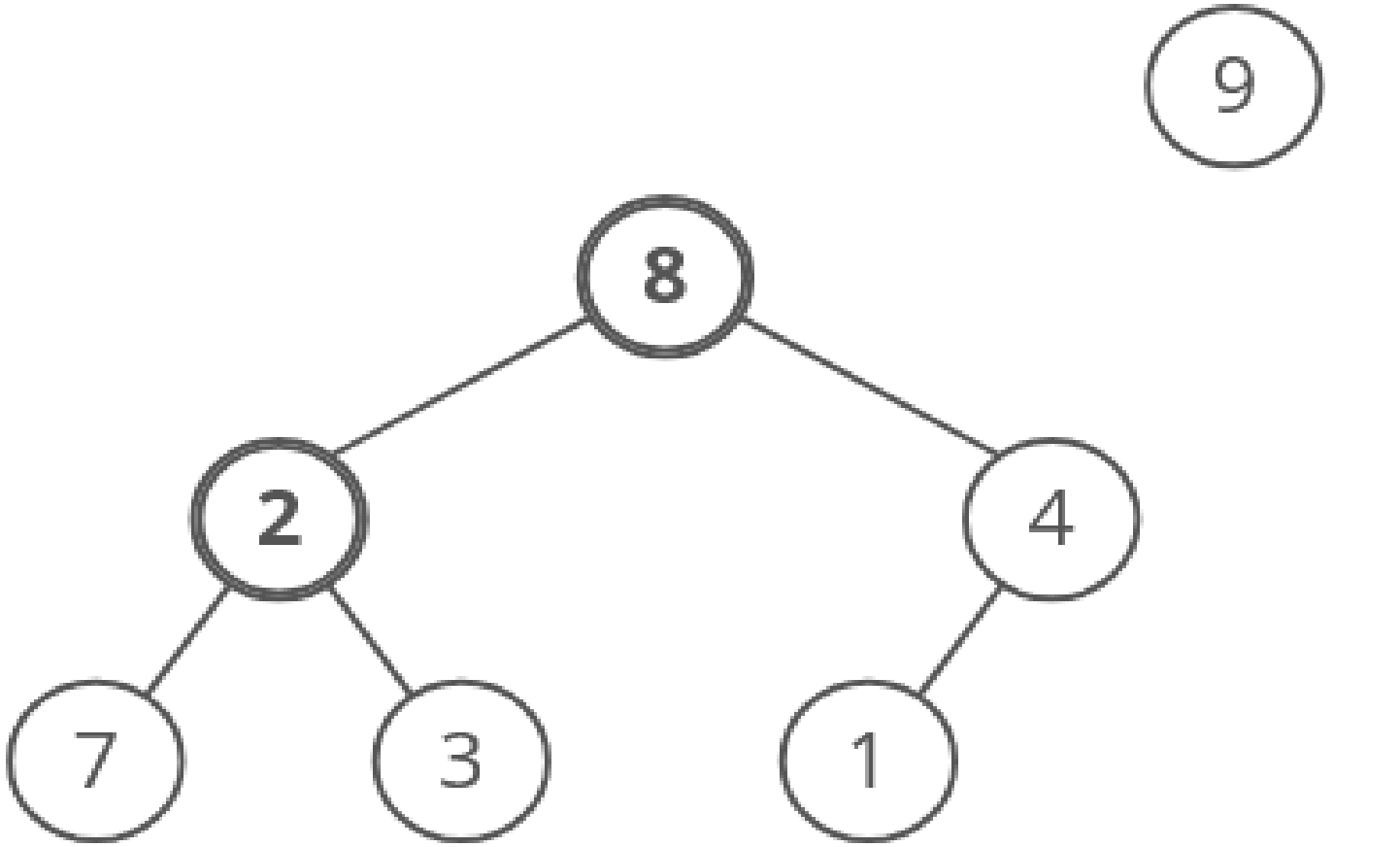
	8	4	7	3	1	2


9
8
4
7
3
1
2
8
4
7
3
1
2

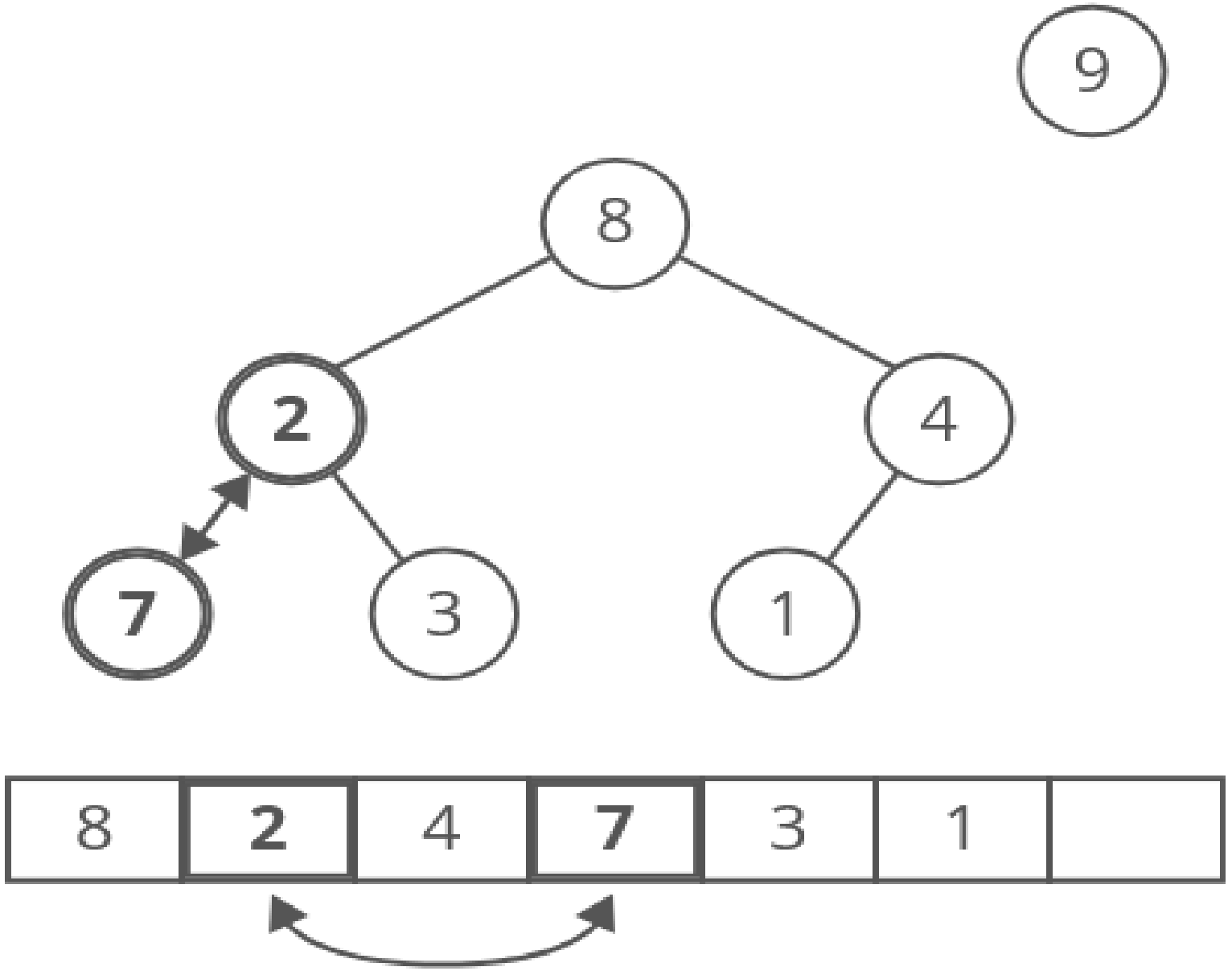


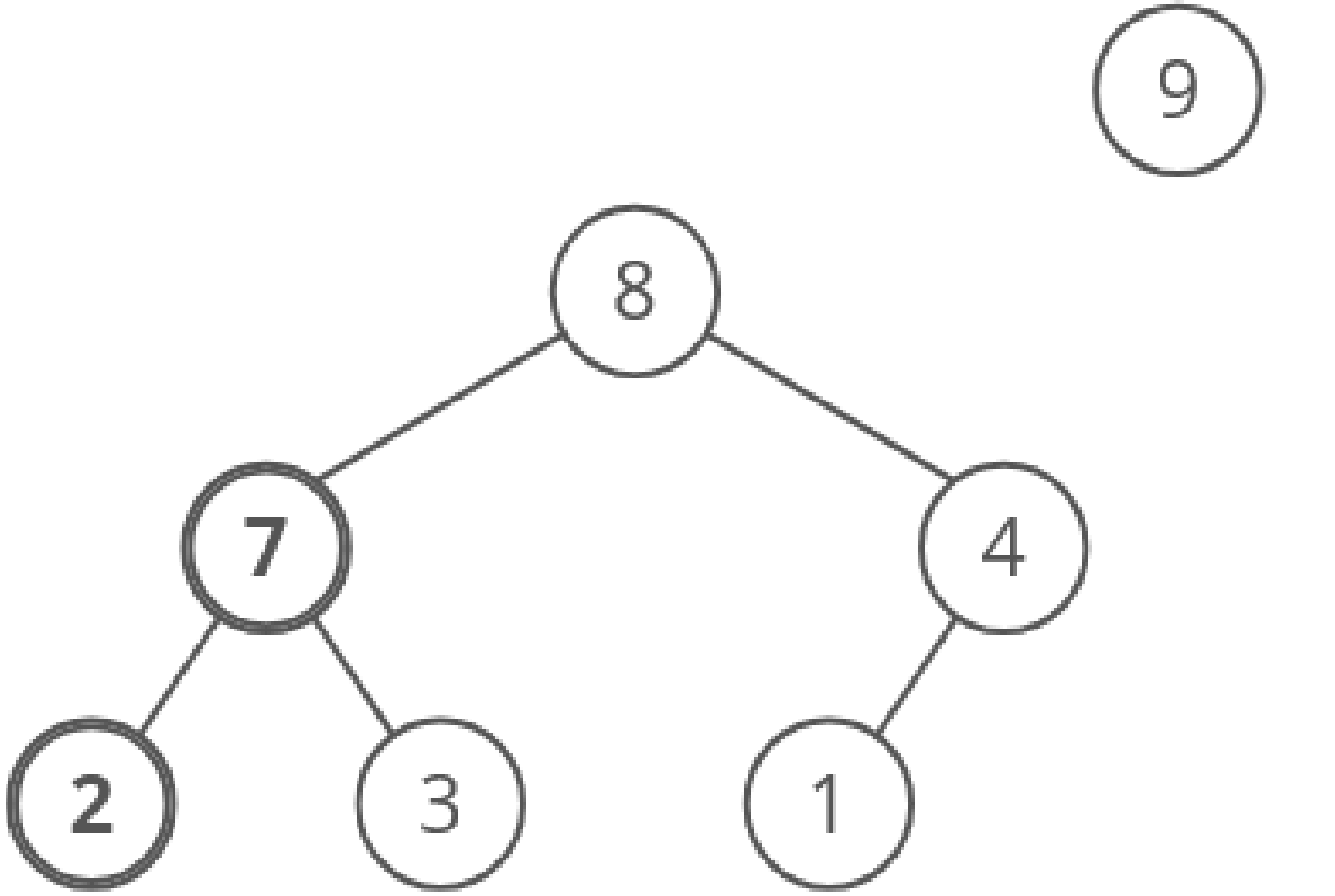
2	8	4	7	3	1	


9
2
8
4
7
3
1
2
8
4
7
3
1

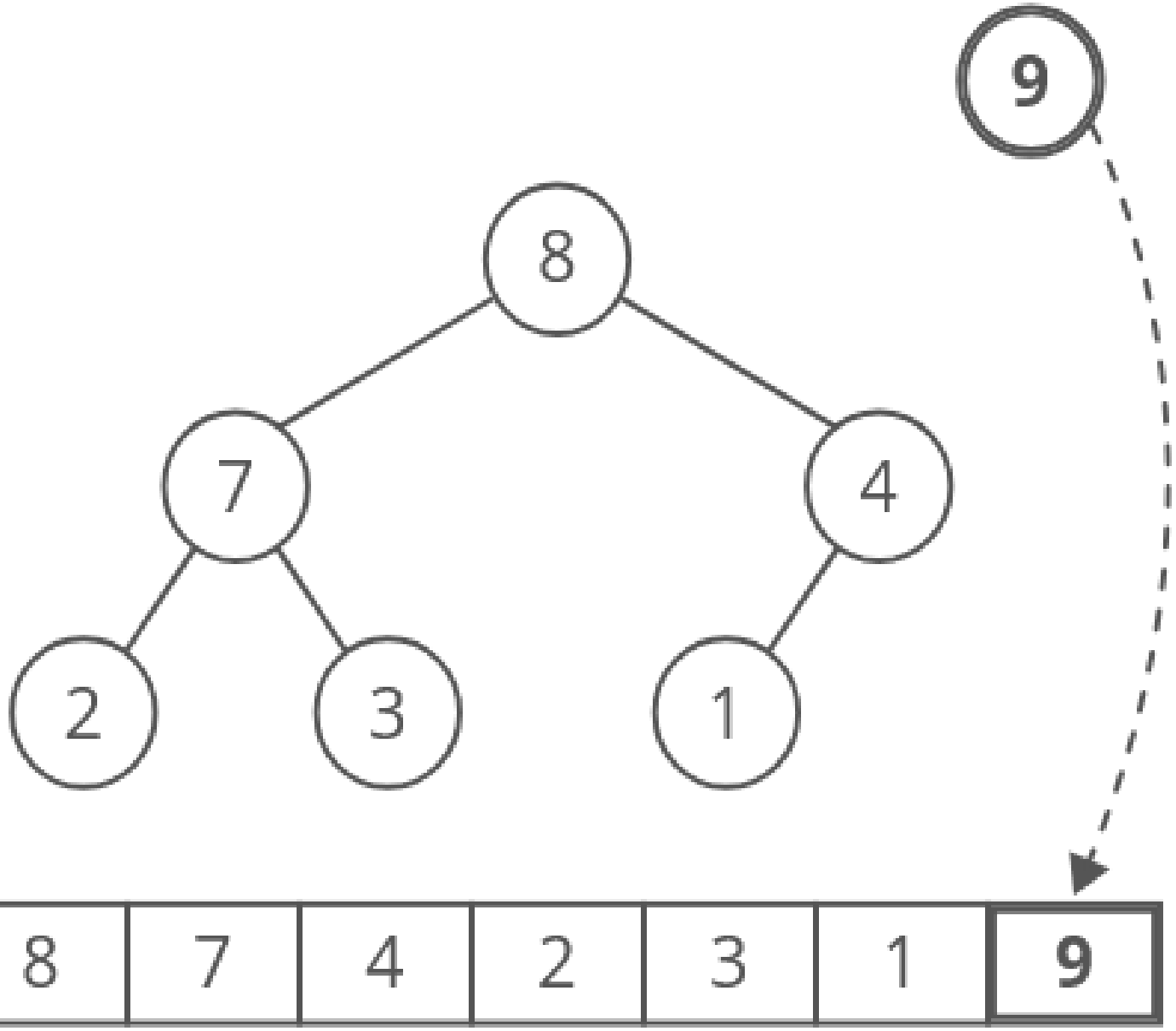


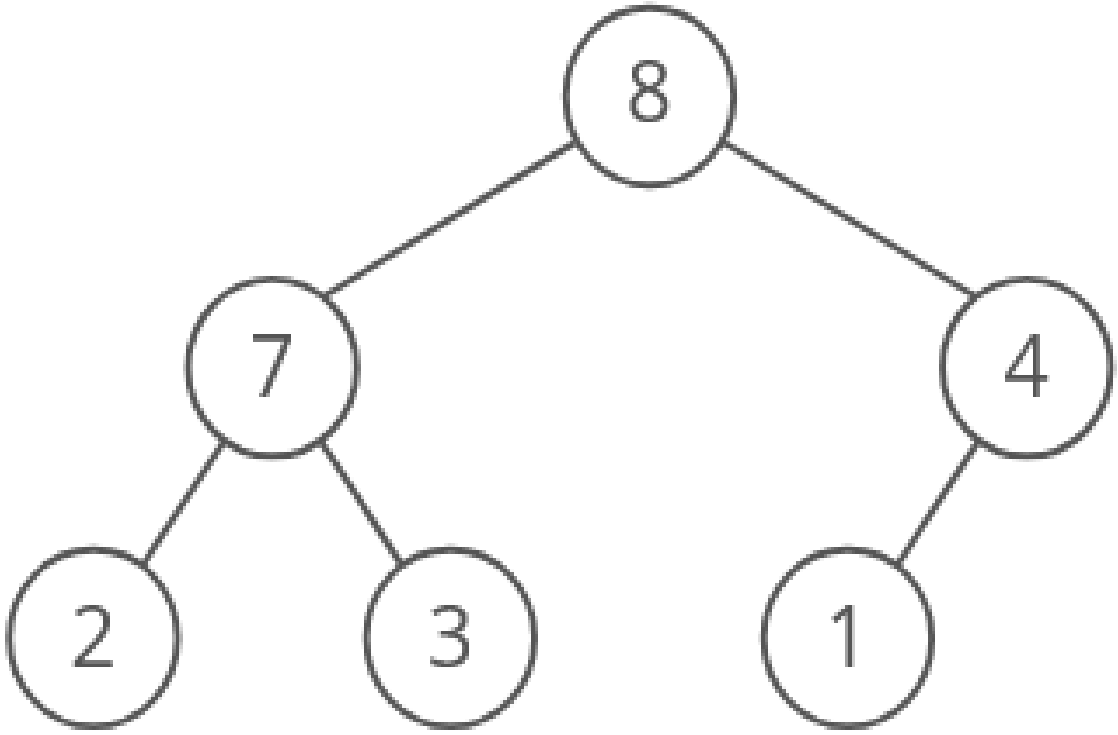
8	2	4	7	3	1	

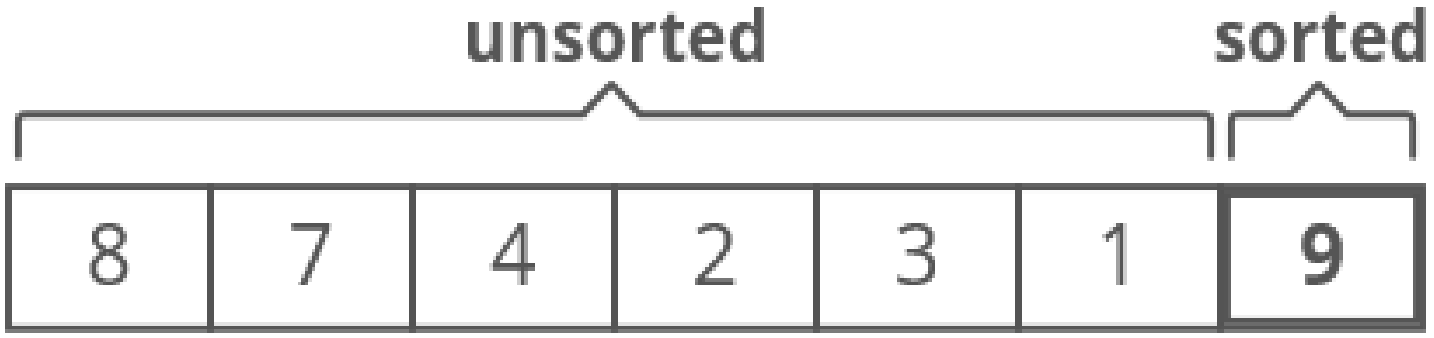

9
8
2
4
7
3
1
8
2
4
7
3
1

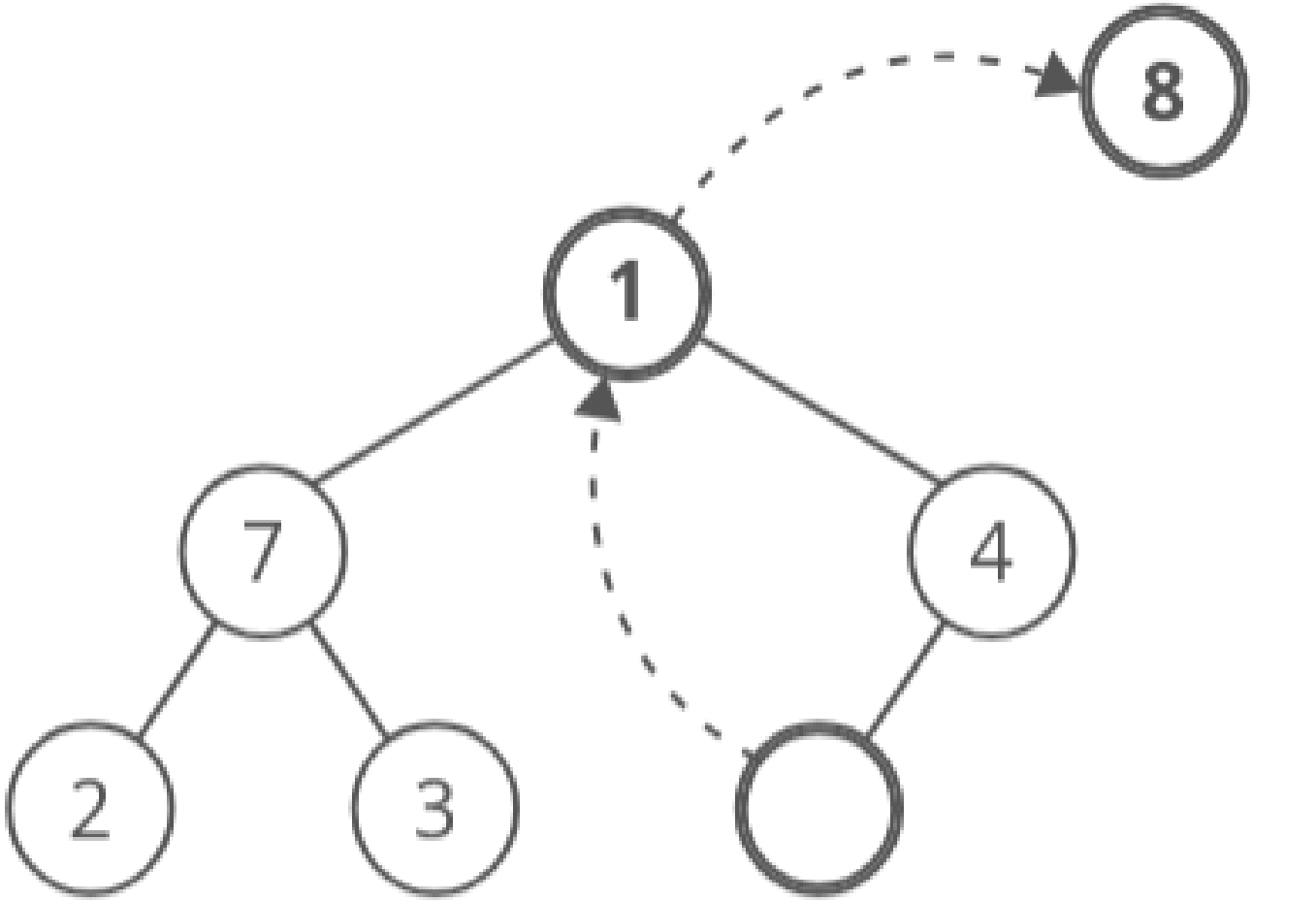


8	**7**	4	**2**	3	1	


9
8
7
4
2
3
1
8
7
4
2
3
1
9


8
7
4
2
3
1


unsorted
sorted
8
7
4
2
3
1
9



1	7	4	2	3		**9**

You can guess the rest…

- https://www.interviewcake.com/concept/java/heapsort

Complexity

- Building the initial heap - $O(n)$

Complexity

- Building the initial heap - $O(n)$
- Deleting from a heap - $O(\log n)$

Complexity

- Building the initial heap - $O(n)$
- Deleting from a heap - $O(\log n)$
- N elements – N deletions - $O(n * \log n)$

The fastest sorting algorithm in the west?

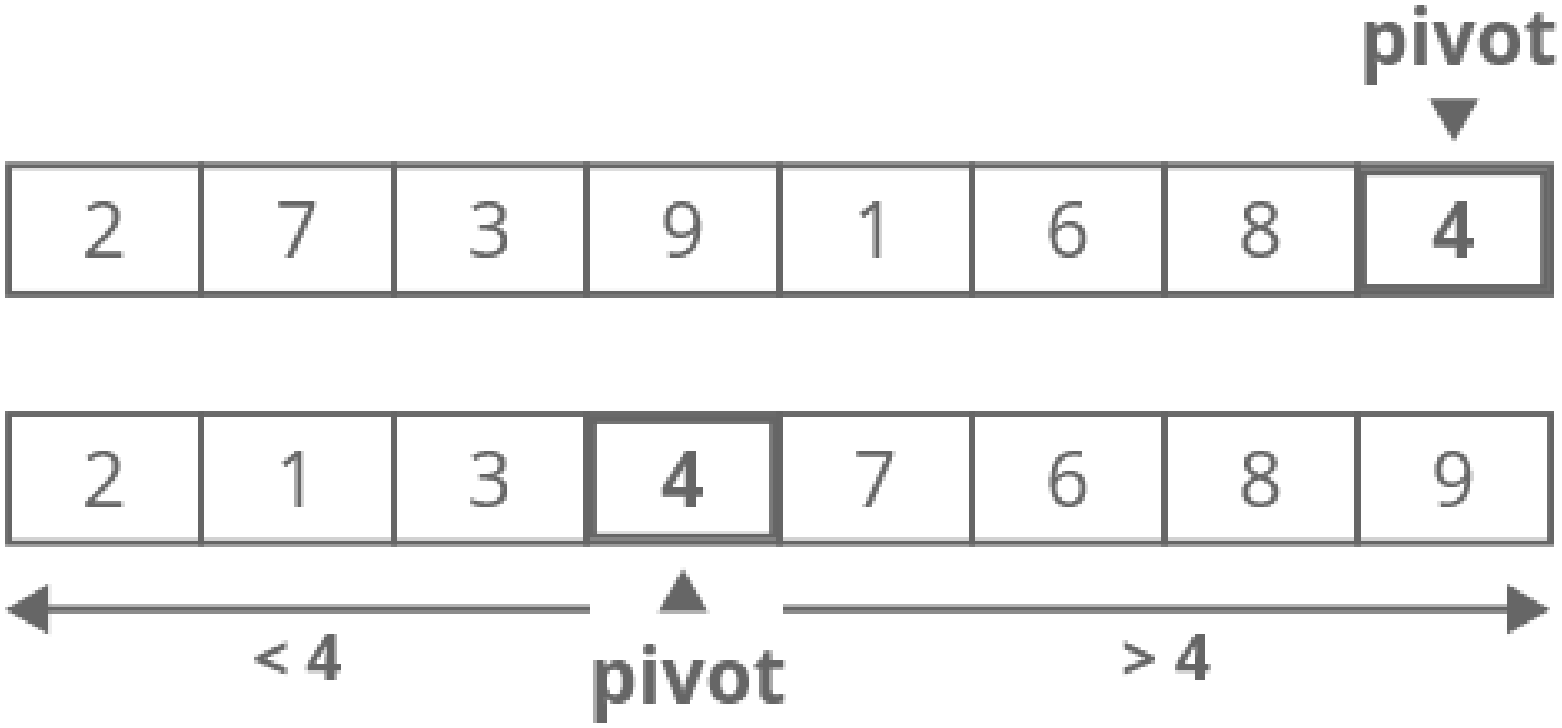


A new foe appeared!

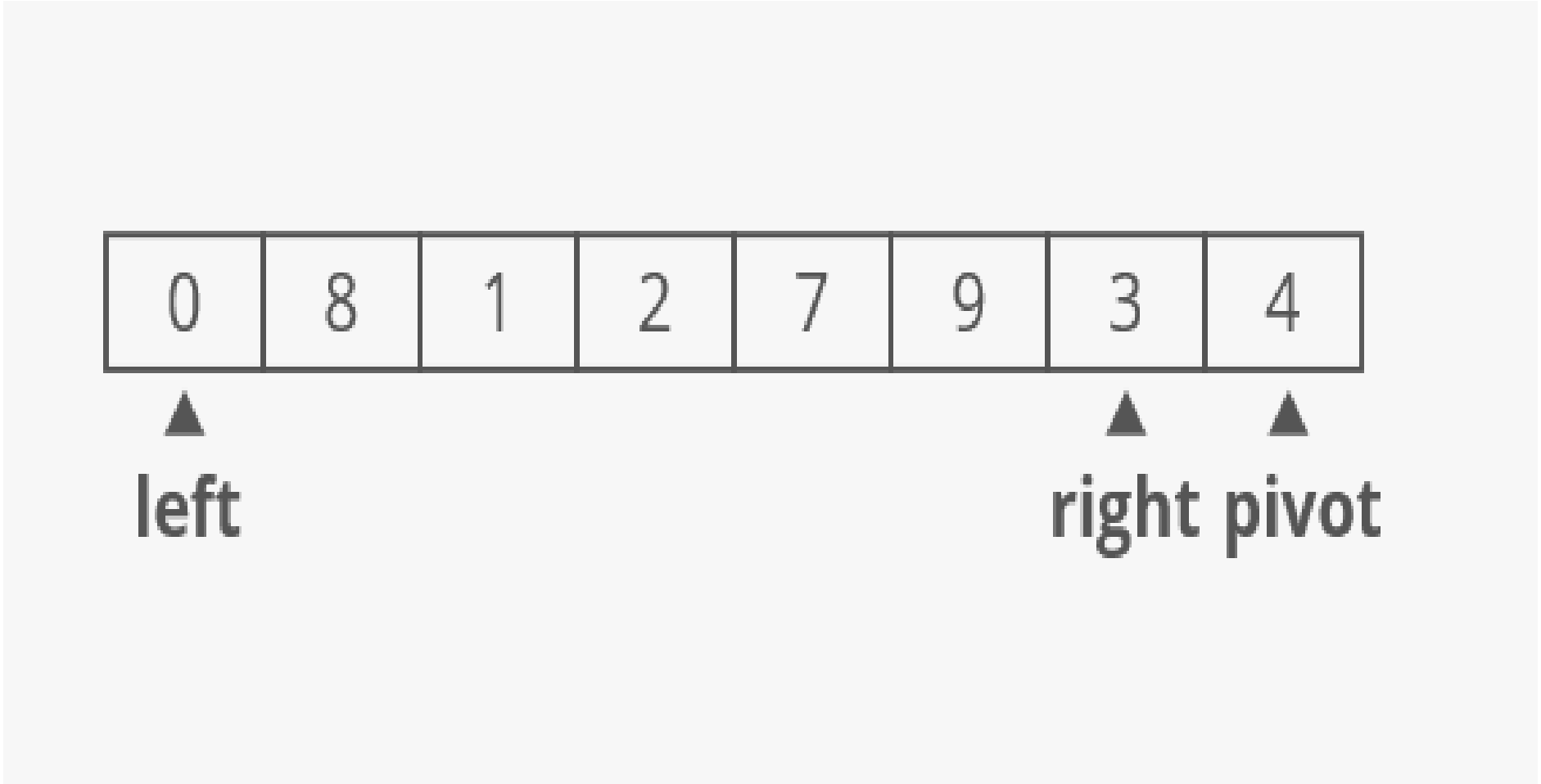


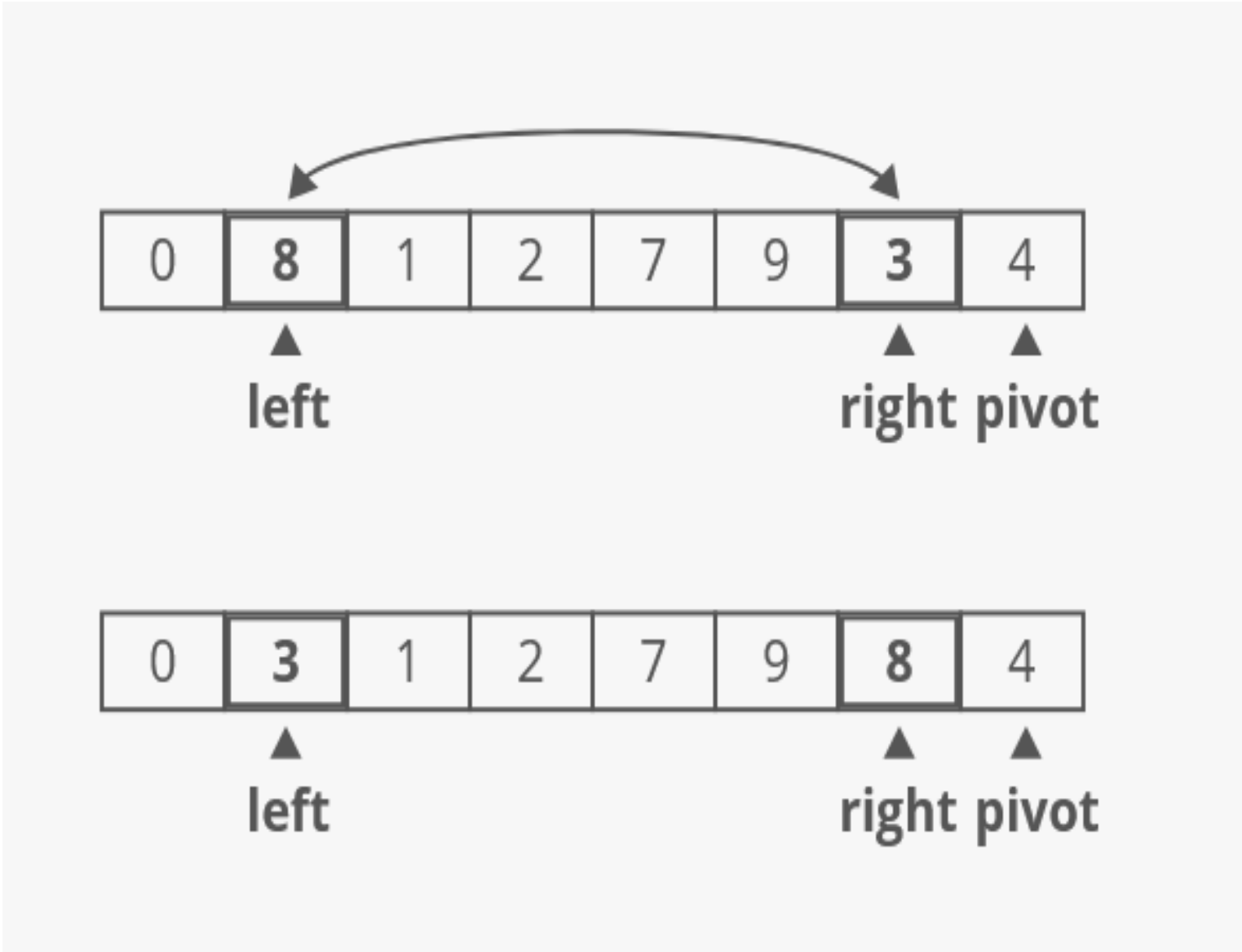
Quicksort

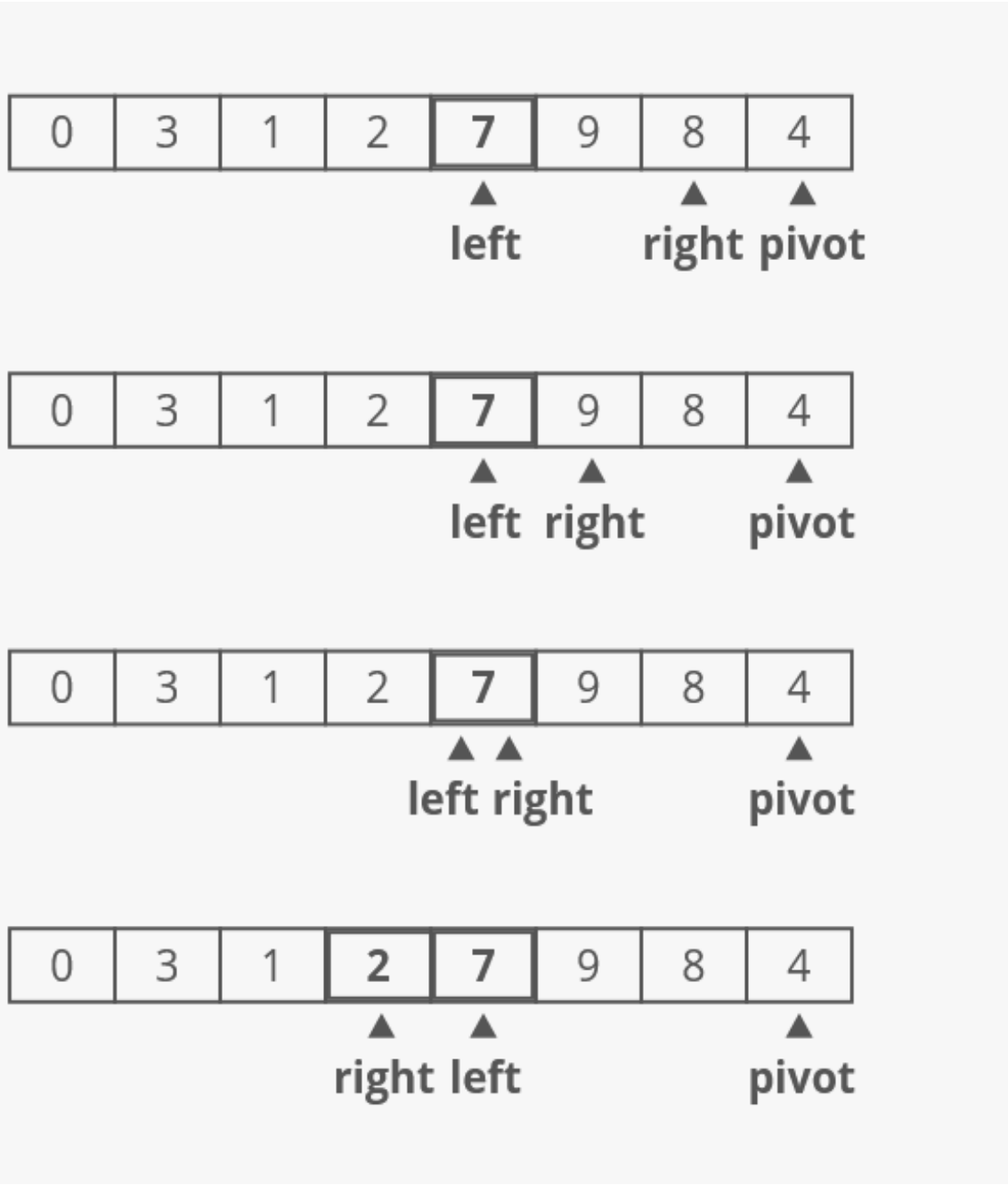
https://www.interviewcake.com/concept/java/quicksort

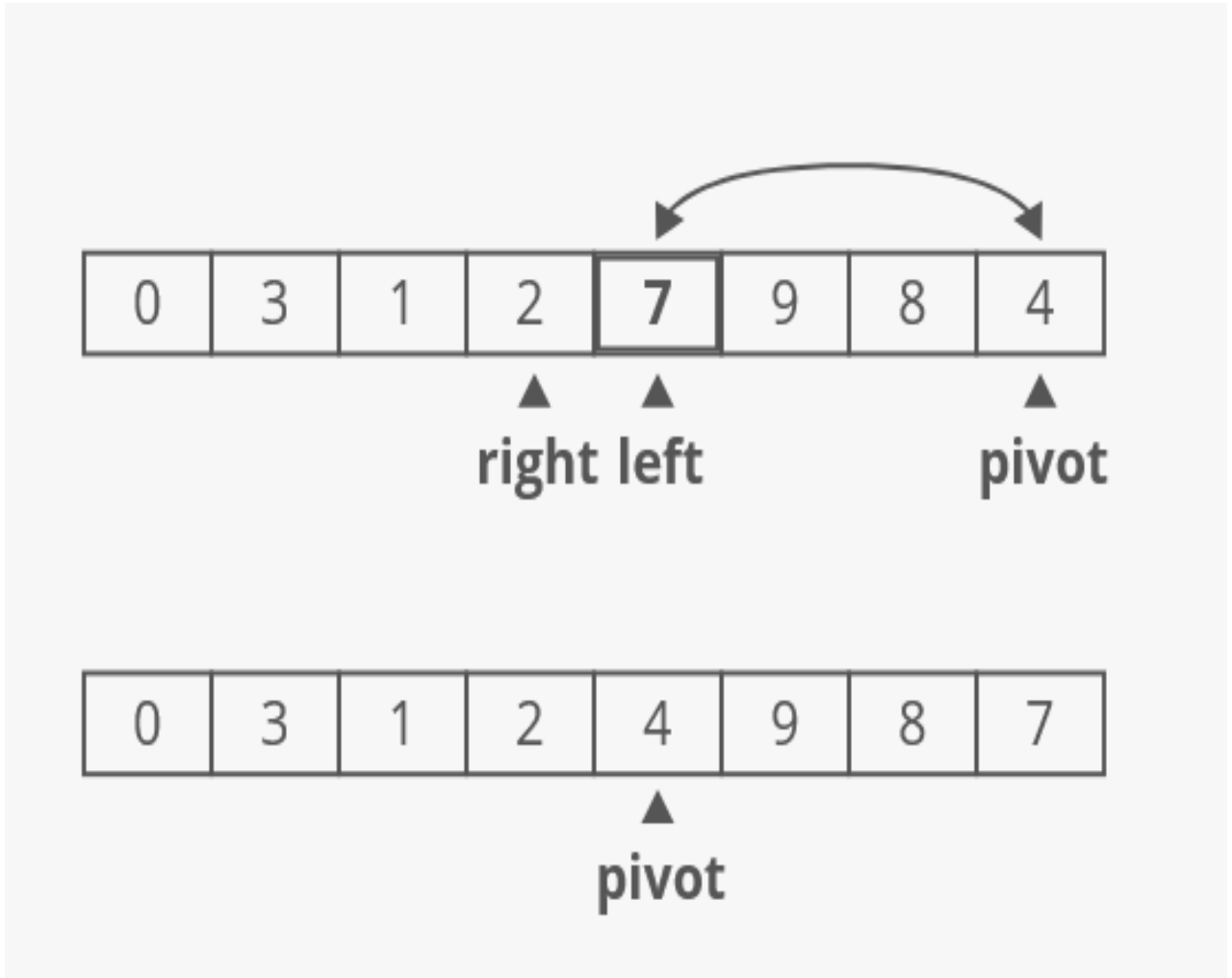


Quicksort

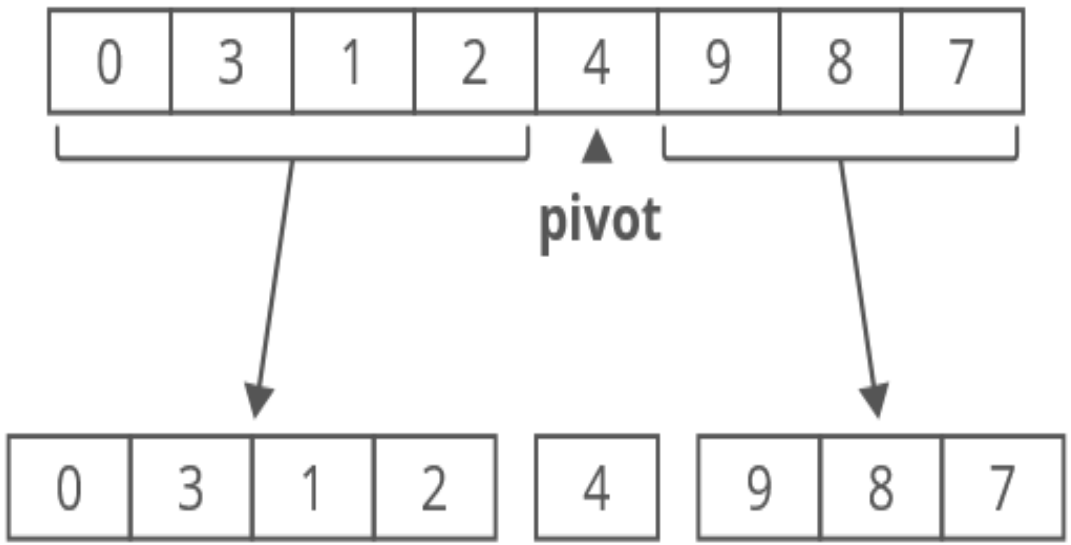



0
8
1
2
7
9
3
4
left
right
pivot
0
3
1
2
7
9
8
4
left
right
pivot


0 3 1 2 7 9 8 4
left
right
pivot
0 3 1 2 7 9 8 4
left
right
pivot
0 3 1 2 7 9 8 4
left
right
pivot
0 3 1 2 7 9 8 4
right
left
pivot


0
3
1
2
7
9
8
4
right
left
pivot
0
3
1
2
4
9
8
7
pivot

Recursively sort the two halves



Time complexity

	Complexity
Worst case time	$O(n^2)$
Best case time	$O(n \lg n)$
Average case time	$O(n \lg n)$

Space Complexity

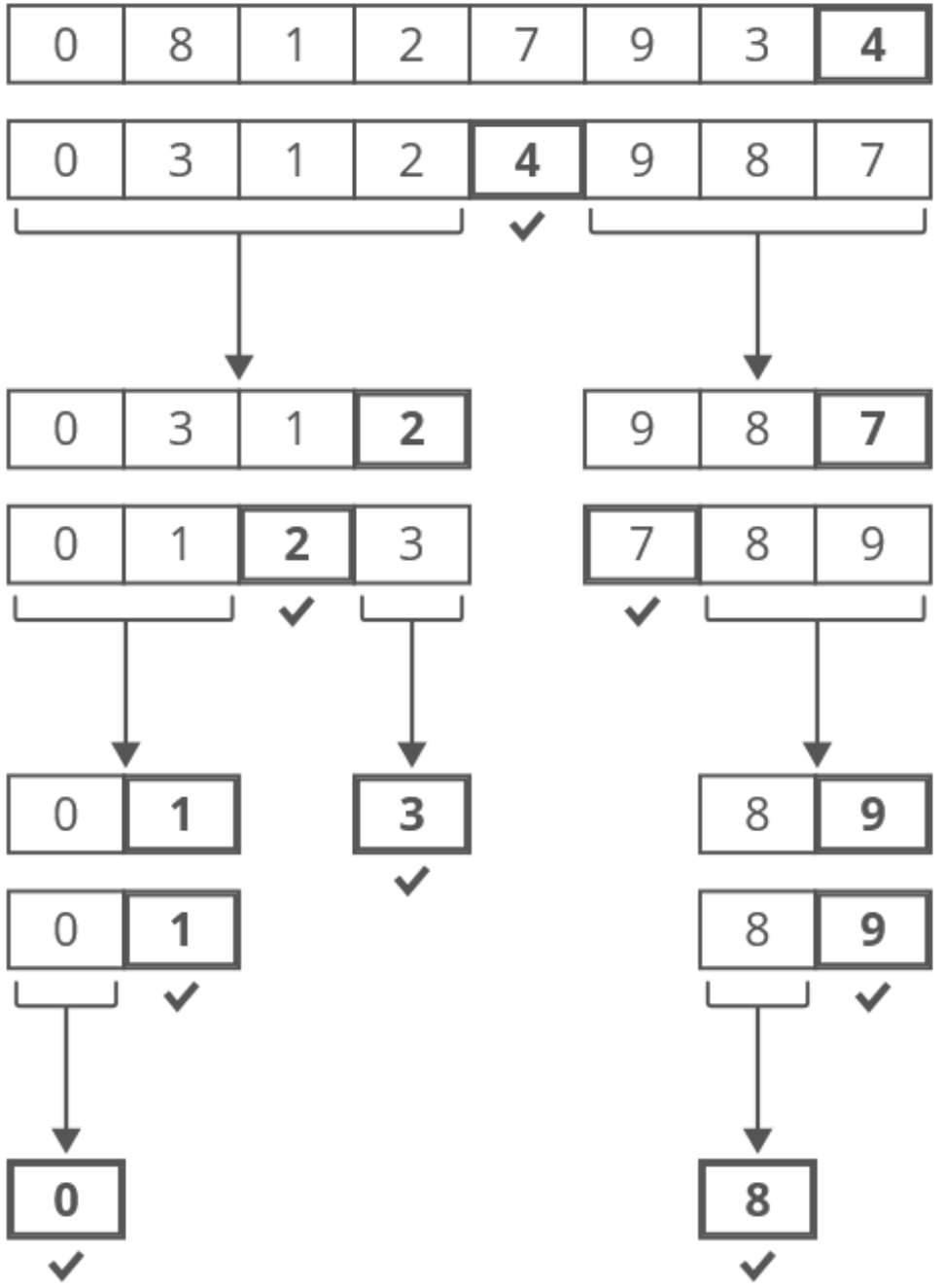
- Recursion allocates a stack frame for each recursive call

Space Complexity

- Recursion allocates a stack frame for each recursive call
- How tall will the recursion tree be?

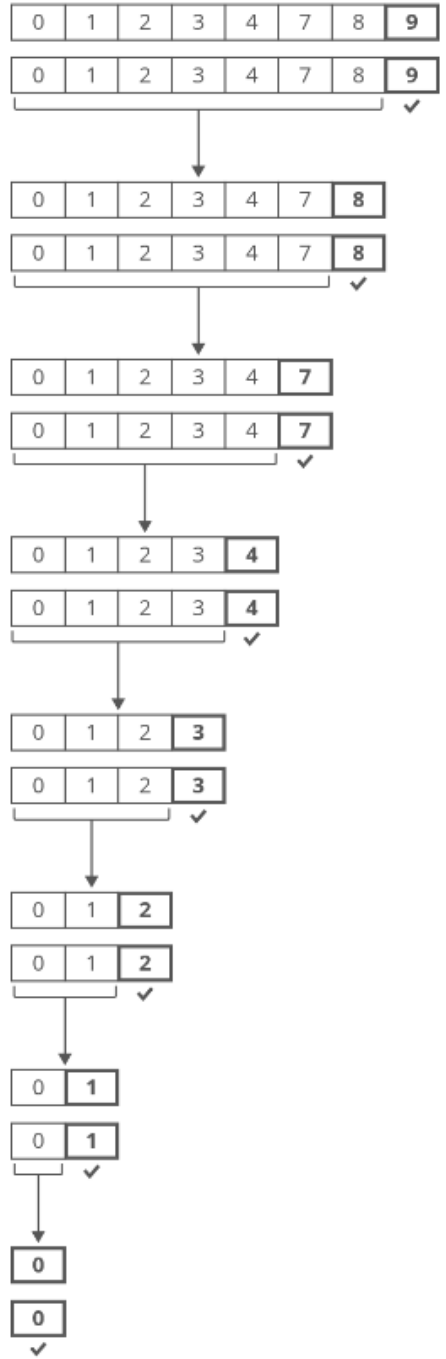
Space Complexity – best case

- The pivot is the *median*
- *Space complexity is O(lg(n))*



Space Complexity – worst case

- *The input array is sorted & the pivot is the last elem.*
- *Space complexity is O(n)*



Who wins?

Heapsort

- Faster in the worst-case scenario
- Slower on average
- **Uses no extra space**

Quicksort

- *Much faster on average*
- Much slower in the worst-case scenario
- **Uses extra space**

Linux uses Heapsort! – F.Y.I

- https://github.com/torvalds/linux/blob/master/lib/sort.c#L53

Kent Overstreet and akpm00 lib/sort.c: add _nonatomic() variants with cond_resched()

Code Blame

```
1   // SPDX-License-Identifier: GPL-2.0
2   /*
3    * A fast, small, non-recursive O(n log n) sort for the Linux kernel
4    *
5    * This performs n*log2(n) + 0.37*n + o(n) comparisons on average,
6    * and 1.5*n*log2(n) + O(n) in the (very contrived) worst case.
7    *
8    * Quicksort manages n*log2(n) - 1.26*n for random inputs (1.63*n
9    * better) at the expense of stack usage and much larger code to avoid
10   * quicksort's O(n^2) worst case.
11   */
12
```